LogicLab Programming Logic and Design

Answers only

229 numbered exercises covering the five learning units and task library. Use the same exercise numbers to match questions with the answer pack. Methods use zero-based arrays and inclusive FOR bounds. Written tasks can have other valid solutions.

Open revision material. Attempt each question before consulting the answer pack.

Unit 1 Algorithms and the development life cycle

Exercise 1 Knowledge check 1

B. Clarify inputs, outputs and requirements
Requirements establish what the solution must achieve before implementation choices are made.

Exercise 2 Knowledge check 2

C. Identify required inputs and outputs
Identify required inputs and outputs. Review the related theory and work through a small example to check the rule.

Exercise 3 Knowledge check 3

B. A finite ordered solution procedure
A finite ordered solution procedure. Review the related theory and work through a small example to check the rule.

Exercise 4 Knowledge check 4

A. Throughout development
Throughout development. Review the related theory and work through a small example to check the rule.

Exercise 5 Knowledge check 5

C. Its outputs still need testing
Its outputs still need testing. Review the related theory and work through a small example to check the rule.

Exercise 6 Find and fix the error

Line 4 - Logic
SET total = price * quantity
The total price is unit price multiplied by the quantity. Adding them does not represent the stated rule.

Exercise 7 Build or trace a solution

```
BEGIN
INPUT weight
INPUT rate
SET charge = weight * rate
OUTPUT charge
END
```

Expected behaviour:
For weight 3 and rate 12, charge is 36. For weight 0 and rate 12, charge is 0.

Review:
Identify weight and rate as inputs.
Multiply the current inputs to obtain charge.
Display the result and test zero as a boundary.

Exercise 8 Apply the concept

SET charge = weight * rate + 15. For weight 3 and rate 12, expect 51; for weight 0, expect 15. Confirm whether zero-weight parcels are actually permitted in the requirements.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 1 Pseudocode and flowchart symbols

Exercise 9 Knowledge check 1

C. Parallelogram
Input and output use parallelograms; a diamond is a decision.

Exercise 10 Knowledge check 2

B. Process rectangle
Process rectangle. Review the related theory and work through a small example to check the rule.

Exercise 11 Knowledge check 3

A. Direction of control flow
Direction of control flow. Review the related theory and work through a small example to check the rule.

Exercise 12 Knowledge check 4

C. To distinguish condition outcomes
To distinguish condition outcomes. Review the related theory and work through a small example to check the rule.

Exercise 13 Knowledge check 5

B. No, state the chosen conventions
No, state the chosen conventions. Review the related theory and work through a small example to check the rule.

Exercise 14 Find and fix the error

Line 5 - Syntax
ENDIF
An IF structure must close with ENDIF in the platform notation. ENDWHILE closes a different structure.

Exercise 15 Build or trace a solution

```
BEGIN
READ temperature
IF temperature &lt; 15 THEN
DISPLAY "Cold"
ELSE
DISPLAY "Mild or warm"
END IF
END
```

Expected behaviour:
14 gives Cold; 15 gives Mild or warm.

Review:
Use a parallelogram for input.
Use a diamond for temperature < 15 and label both branches.
Join the paths after their output symbols.

Exercise 16 Apply the concept

Reading: parallelogram. Computing: process rectangle. Testing: decision diamond with labelled branches. Ending: rounded terminal. A hierarchy chart would not show these execution paths.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 1 Data types, variables and constants

Exercise 17 Knowledge check 1

B. STRING
A phone number is an identifier, not a quantity; a string preserves leading zeroes and formatting.

Exercise 18 Knowledge check 2

A. Gives a variable its starting value
Gives a variable its starting value. Review the related theory and work through a small example to check the rule.

Exercise 19 Knowledge check 3

C. BOOLEAN
BOOLEAN. Review the related theory and work through a small example to check the rule.

Exercise 20 Knowledge check 4

B. 1
1. Review the related theory and work through a small example to check the rule.

Exercise 21 Knowledge check 5

A. Yes, a negative age can violate requirements
Yes, a negative age can violate requirements. Review the related theory and work through a small example to check the rule.

Exercise 22 Find and fix the error

Line 3 - Initialisation
SET total = 0, then INPUT price
Before accumulation, initialise total to zero. Keep the original INPUT price after the initialisation.

Exercise 23 Build or trace a solution

```
BEGIN
DECLARE product AS REAL
product = 1
FOR i = 1 TO 3
INPUT value
product = product * value
NEXT i
OUTPUT product
END
```

Expected behaviour:
Inputs 2, 3, 4 give 24. Inputs 2, 0, 4 give 0.

Review:
Use REAL because inputs may have fractional parts.
Initialise multiplication to 1, not 0.
Multiply each value into product once.

Exercise 24 Apply the concept

Student number: STRING, to preserve leading zeroes. Active flag: BOOLEAN. Whole-item count: INTEGER. Average mark: REAL. Validate allowed values separately from choosing types.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 1 Expressions and operator precedence

Exercise 25 Knowledge check 1

B. 10
Multiplication is performed before addition, so 3 * 2 is 6 and 4 + 6 is 10.

Exercise 26 Knowledge check 2

C. 4
4. Review the related theory and work through a small example to check the rule.

Exercise 27 Knowledge check 3

B. 2
2. Review the related theory and work through a small example to check the rule.

Exercise 28 Knowledge check 4

A. 3
3. Review the related theory and work through a small example to check the rule.

Exercise 29 Knowledge check 5

C. (a + b) / 2
(a + b) / 2. Review the related theory and work through a small example to check the rule.

Exercise 30 Find and fix the error

Line 4 - Calculation
SET average = (mark1 + mark2) / 2
Parentheses make the addition happen before dividing the total by two.

Exercise 31 Build or trace a solution

```
BEGIN
INPUT price
INPUT quantity
SET subtotal = price * quantity
SET total = subtotal * (1 - 0.10)
OUTPUT total
END
```

Expected behaviour:
Price 20, quantity 3: subtotal 60 and total 54.

Review:
Calculate subtotal before applying the discount.
A 10% discount leaves 90% of subtotal.
Test no items and a simple positive quantity.

Exercise 32 Apply the concept

hours = 135 DIV 60 gives 2; minutes = 135 MOD 60 gives 15. DIV and MOD are expressions displayed by the converter, not executed by it.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 1 Structured design and trace tables

Exercise 33 Knowledge check 1

C. Repetition
Repetition is the structure for repeated execution.

Exercise 34 Knowledge check 2

B. Record changing state and output
Record changing state and output. Review the related theory and work through a small example to check the rule.

Exercise 35 Knowledge check 3

A. 7
7. Review the related theory and work through a small example to check the rule.

Exercise 36 Knowledge check 4

C. IF/ELSE
IF/ELSE. Review the related theory and work through a small example to check the rule.

Exercise 37 Knowledge check 5

B. Clear names and focused responsibilities
Clear names and focused responsibilities. Review the related theory and work through a small example to check the rule.

Exercise 38 Find and fix the error

Line 5 - Logic
OUTPUT x
The requirement is to display the final x. Here y is 5 and the final x is 10.

Exercise 39 Build or trace a solution

```
BEGIN
SET a = 4
SET b = a + 3
SET a = b - 2
SET b = a * 2
OUTPUT a, b
END
```

Expected behaviour:
After assignments: (a=4,b=unset), (4,7), (5,7), (5,10). Output: 5, 10.

Review:
Assignment changes only the variable on its left.
Use the latest values on the right-hand side.
Distinguish the current state from displayed output.

Exercise 40 Apply the concept

Reading the two marks in order is sequence. Choosing a result is selection. Processing ten learners uses repetition. A complete program can combine all three structures.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 2 Boolean expressions and basic selection

Exercise 41 Knowledge check 1

A. When both are false
Inclusive OR is false only when neither operand is true.

Exercise 42 Knowledge check 2

A. Only when both are true
Only when both are true. Review the related theory and work through a small example to check the rule.

Exercise 43 Knowledge check 3

C. TRUE
TRUE. Review the related theory and work through a small example to check the rule.

Exercise 44 Knowledge check 4

B. mark >= 0 AND mark <= 100
mark >= 0 AND mark <= 100. Review the related theory and work through a small example to check the rule.

Exercise 45 Knowledge check 5

A. 49, 50 and 51
49, 50 and 51. Review the related theory and work through a small example to check the rule.

Exercise 46 Find and fix the error

Line 3 - Boundary
IF mark >= 50 THEN
A score of exactly 50 must pass. The >= operator includes that boundary.

Exercise 47 Build or trace a solution

```
BEGIN
INPUT mark
IF mark &gt;= 0 AND mark &lt;= 100 THEN
OUTPUT "Valid"
ELSE
OUTPUT "Invalid"
ENDIF
END

Expected behaviour:
-1 invalid, 0 valid, 100 valid, 101 invalid.

Review:
Both the lower and upper bounds must hold.
Use AND for the intersection of the two conditions.
Check values at and just outside each endpoint.
```

Exercise 48 Apply the concept

```
Invalid: mark &lt; 0 OR mark &gt; 100. Equivalent: NOT (mark &gt;= 0 AND mark &lt;= 100). Parentheses
 make the negated range explicit.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.
```

Unit 2 Precedence and nested decisions

Exercise 49 Knowledge check 1

B. TRUE
AND is evaluated before OR: TRUE OR (FALSE AND FALSE) is TRUE.

Exercise 50 Knowledge check 2

C. NOT
NOT. Review the related theory and work through a small example to check the rule.

Exercise 51 Knowledge check 3

B. Only after execution enters its outer branch
Only after execution enters its outer branch. Review the related theory and work through a small example to check the rule.

Exercise 52 Knowledge check 4

A. The more specific higher band should be checked first
The more specific higher band should be checked first. Review the related theory and work through a small example to check the rule.

Exercise 53 Knowledge check 5

C. One expression against listed values
One expression against listed values. Review the related theory and work through a small example to check the rule.

Exercise 54 Find and fix the error

Line 4 - Logic
IF age >= 18 AND hasID = TRUE THEN
Both conditions are required. OR would allow someone who meets only one condition.

Exercise 55 Build or trace a solution

```
BEGIN
INPUT mark
IF mark &gt;= 75 THEN
OUTPUT "Distinction"
ELSEIF mark &gt;= 50 THEN
OUTPUT "Pass"
ELSE
OUTPUT "Revise"
END IF
END

Expected behaviour:
75 -&gt; Distinction, 74 -&gt; Pass, 50 -&gt; Pass, 49 -&gt; Revise.

Review:
Test the highest band first.
ELSEIF is considered only after the preceding condition is false.
Check each exact threshold and the value immediately below.
```

Exercise 56 Apply the concept

```
IF mark &gt;= 75 THEN
  OUTPUT "Distinction"
ELSE
  IF mark &gt;= 50 THEN
    OUTPUT "Pass"
  ELSE
    OUTPUT "Revise"
  ENDIF
ENDIF

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.
```

Unit 2 Counted loops and accumulators

Exercise 57 Knowledge check 1

B. 4
Both endpoints are included: i is 1, 2, 3 and 4.

Exercise 58 Knowledge check 2

B. 1
1. Review the related theory and work through a small example to check the rule.

Exercise 59 Knowledge check 3

A. After accumulation finishes
After accumulation finishes. Review the related theory and work through a small example to check the rule.

Exercise 60 Knowledge check 4

C. 3
3. Review the related theory and work through a small example to check the rule.

Exercise 61 Knowledge check 5

B. 5
5. Review the related theory and work through a small example to check the rule.

Exercise 62 Find and fix the error

Line 5 - Logic
SET total = total + value
Assigning value alone discards the previous total. An accumulator adds the new value to the running total.

Exercise 63 Build or trace a solution

```
BEGIN
SET passes = 0
FOR i = 1 TO 4
INPUT mark
IF mark &gt;= 50 THEN
SET passes = passes + 1
ENDIF
ENDFOR
OUTPUT passes
END
```

Expected behaviour:
Inputs 40, 50, 70, 20 give 2 passes.

Review:
Initialise the counter once before the loop.
Increment by one only when the condition is true.
Display passes after all four inputs.

Exercise 64 Apply the concept

```
BEGIN
SET total = 0
FOR i = 1 TO 4
INPUT mark
SET total = total + mark
ENDFOR
SET average = total / 4
OUTPUT average
END
```

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 2 Conditional loops and priming input

Exercise 65 Knowledge check 1

B. REPEAT/UNTIL
REPEAT/UNTIL tests after executing its body.

Exercise 66 Knowledge check 2

A. Yes
Yes. Review the related theory and work through a small example to check the rule.

Exercise 67 Knowledge check 3

C. It should normally be excluded from data totals
It should normally be excluded from data totals. Review the related theory and work through a small example to check the rule.

Exercise 68 Knowledge check 4

B. When its condition is true
When its condition is true. Review the related theory and work through a small example to check the rule.

Exercise 69 Knowledge check 5

A. To avoid division by zero
To avoid division by zero. Review the related theory and work through a small example to check the rule.

Exercise 70 Find and fix the error

Line 5 - Logic
INPUT value
A new input allows the condition to change. Assigning value to itself cannot obtain the next value.

Exercise 71 Build or trace a solution

```
BEGIN
SET total = 0
SET count = 0
INPUT value
WHILE value &lt;&gt; -1 DO
SET total = total + value
SET count = count + 1
INPUT value
ENDWHILE
IF count &gt; 0 THEN
OUTPUT total / count
ELSE
OUTPUT "No data"
ENDIF
END
```

Expected behaviour:
10, 20, -1 gives 15. The input -1 alone gives No data.

Review:
Read the priming value before checking the condition.
Exclude the sentinel from total and count.
Guard division so count cannot be zero.

Exercise 72 Apply the concept

```
REPEAT
  INPUT age
UNTIL age &gt;= 0
OUTPUT age
```

The loop runs at least once and continues while invalid input is supplied.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 2 Nested loops

Exercise 73 Knowledge check 1

C. 6
Each of the two outer iterations runs three inner iterations: 2 * 3 = 6.

Exercise 74 Knowledge check 2

C. 12
12. Review the related theory and work through a small example to check the rule.

Exercise 75 Knowledge check 3

B. At the start of each outer iteration
At the start of each outer iteration. Review the related theory and work through a small example to check the rule.

Exercise 76 Knowledge check 4

A. So the inner loop does not overwrite the outer state
So the inner loop does not overwrite the outer state. Review the related theory and work through a small example to check the rule.

Exercise 77 Knowledge check 5

C. 10
10. Review the related theory and work through a small example to check the rule.

Exercise 78 Find and fix the error

Line 3 - Logic
FOR column = 1 TO 3
Use a distinct inner counter, such as column. Adjust the body to use row and column as required.

Exercise 79 Build or trace a solution

```
BEGIN
FOR row = 1 TO 3
FOR column = 1 TO 2
SET product = row * column
OUTPUT product
ENDFOR
ENDFOR
END
```

Expected behaviour:
Outputs in order: 1, 2, 2, 4, 3, 6.

Review:
Keep row and column counters separate.
For each row, complete both column iterations.
Multiply the number of iterations to check the six outputs.

Exercise 80 Apply the concept

Initialise grandTotal once before both loops. Initialise rowTotal at the start of each outer-loop iteration. Add each cell to both totals inside the inner loop; output each row total after the inner loop and the grand total after the outer loop.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 3 Arrays and element access

Exercise 81 Knowledge check 1

B. 4
Indices are 0, 1, 2, 3 and 4.

Exercise 82 Knowledge check 2

B. 0
0. Review the related theory and work through a small example to check the rule.

Exercise 83 Knowledge check 3

A. 0 TO n - 1
0 TO n - 1. Review the related theory and work through a small example to check the rule.

Exercise 84 Knowledge check 4

C. values[r][c]
values[r][c]. Review the related theory and work through a small example to check the rule.

Exercise 85 Knowledge check 5

B. Capacity can exceed the number of used values
Capacity can exceed the number of used values. Review the related theory and work through a small example to check the rule.

Exercise 86 Find and fix the error

Line 2 - Boundary
FOR i = 0 TO 4
For an array of length five, index five is out of bounds. The last valid index is four.

Exercise 87 Build or trace a solution

```
BEGIN
SET total = 0
FOR i = 0 TO 4
SET total = total + values[i]
ENDFOR
OUTPUT total
END
```

Expected behaviour:
For [2,4,6,8,10], output 30. The last accessed index is 4.

Review:
The meaningful indices run from 0 to length - 1.
Initialise total before visiting any element.
Use each index exactly once.

Exercise 88 Apply the concept

```
FOR row = 0 TO 1
  FOR column = 0 TO 2
    OUTPUT values[row][column]
  ENDFOR
ENDFOR
```

There are six element accesses.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 3 Linear search

Exercise 89 Knowledge check 1

B. It distinguishes not found from index 0
-1 is outside the valid indices and can represent no match.

Exercise 90 Knowledge check 2

A. No
No. Review the related theory and work through a small example to check the rule.

Exercise 91 Knowledge check 3

C. n
n. Review the related theory and work through a small example to check the rule.

Exercise 92 Knowledge check 4

B. Continue through all valid positions
Continue through all valid positions. Review the related theory and work through a small example to check the rule.

Exercise 93 Knowledge check 5

A. Check that a match was found
Check that a match was found. Review the related theory and work through a small example to check the rule.

Exercise 94 Find and fix the error

Line 2 - Initialisation
SET foundAt = -1
Zero is a valid position, so it cannot unambiguously mean not found. Use -1 for this zero-based example.

Exercise 95 Build or trace a solution

```
BEGIN
SET foundAt = -1
SET i = 0
WHILE i &lt; 4 AND foundAt = -1 DO
IF values[i] = target THEN
SET foundAt = i
ENDIF
SET i = i + 1
ENDWHILE
OUTPUT foundAt
END
```

Expected behaviour:
Array [7,3,7,2], target 7 -> 0. Target 9 -> -1.

Review:
Use -1 for not found because 0 is a valid index.
Check the bound before accessing values[i].
Stop when foundAt changes from -1.

Exercise 96 Apply the concept

Traverse all valid indices. Whenever values[i] = target, output or record i. Do not stop after the first match. For [7,3,7,2] and target 7, the positions are 0 and 2.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 3 Parallel arrays

Exercise 97 Knowledge check 1

B. names[2]
A shared index connects attributes belonging to the same record.

Exercise 98 Knowledge check 2

C. A shared record index
A shared record index. Review the related theory and work through a small example to check the rule.

Exercise 99 Knowledge check 3

B. Its related attributes must move too
Its related attributes must move too. Review the related theory and work through a small example to check the rule.

Exercise 100 Knowledge check 4

A. Objects or records holding related attributes
Objects or records holding related attributes. Review the related theory and work through a small example to check the rule.

Exercise 101 Knowledge check 5

C. scores[1]
scores[1]. Review the related theory and work through a small example to check the rule.

Exercise 102 Find and fix the error

Line 3 - Logic
OUTPUT names[i], scores[i]
Use the same index for related attributes; scores[0] may belong to another learner.

Exercise 103 Build or trace a solution

```
BEGIN
SET foundAt = -1
FOR i = 0 TO 2
IF names[i] = targetName THEN
SET foundAt = i
ENDIF
ENDFOR
IF foundAt &lt;&gt; -1 THEN
OUTPUT scores[foundAt]
ELSE
OUTPUT "Not found"
ENDIF
END
```

Expected behaviour:
names=[Lebo,Mpho,Ana], scores=[65,80,72], targetName=Mpho -> 80.

Review:
Use the name array to locate an index.
Guard against the not-found result.
Use the same index in scores.

Exercise 104 Apply the concept

Swap names[j] with names[j+1] using a temporary name. Also swap scores[j] with scores[j+1] using a temporary score. Both swaps must happen together to preserve the name-mark relationship.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 3 Bubble sort

Exercise 105 Knowledge check 1

A. To preserve a value before overwriting it
Without temporary storage, the first overwritten value can be lost.

Exercise 106 Knowledge check 2

B. At the end of the unsorted part
At the end of the unsorted part. Review the related theory and work through a small example to check the rule.

Exercise 107 Knowledge check 3

A. n - 1
n - 1. Review the related theory and work through a small example to check the rule.

Exercise 108 Knowledge check 4

C. values[j] > values[j + 1]
values[j] > values[j + 1]. Review the related theory and work through a small example to check the rule.

Exercise 109 Knowledge check 5

B. Stop after a pass with no swaps
Stop after a pass with no swaps. Review the related theory and work through a small example to check the rule.

Exercise 110 Find and fix the error

Line 4 - Logic
SET values[j + 1] = temp
The original left value is in temp. Using values[j] again duplicates the right value.

Exercise 111 Build or trace a solution

```
BEGIN
FOR j = 0 TO 2
IF values[j] &gt; values[j + 1] THEN
SET temp = values[j]
SET values[j] = values[j + 1]
SET values[j + 1] = temp
ENDIF
ENDFOR
OUTPUT values
END

Expected behaviour:
[4,1,3,2] becomes [1,3,2,4] after one pass. It is not fully sorted yet.

Review:
Compare adjacent values in order.
Swap only when the left value exceeds the right.
The largest value reaches the last position after this full pass.
```

Exercise 112 Apply the concept

```
FOR pass = 0 TO n - 2
  FOR j = 0 TO n - 2 - pass
    IF values[j] &gt; values[j + 1] THEN
      SET temp = values[j]
      SET values[j] = values[j + 1]
      SET values[j + 1] = temp
    ENDIF
  ENDFOR
ENDFOR

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.
```

Unit 4 Decomposition and hierarchy charts

Exercise 113 Knowledge check 1

B. Module organisation and calls
A hierarchy chart describes the module structure and call relationships.

Exercise 114 Knowledge check 2

A. Separate focused responsibilities
Separate focused responsibilities. Review the related theory and work through a small example to check the rule.

Exercise 115 Knowledge check 3

C. How closely a module’s tasks belong together
How closely a module’s tasks belong together. Review the related theory and work through a small example to check the rule.

Exercise 116 Knowledge check 4

B. No, it shows module organisation
No, it shows module organisation. Review the related theory and work through a small example to check the rule.

Exercise 117 Knowledge check 5

A. Dependencies between modules
Dependencies between modules. Review the related theory and work through a small example to check the rule.

Exercise 118 Find and fix the error

Line 5 - Syntax
CALL showResult()
The call name must match the defined method name in this example.

Exercise 119 Build or trace a solution

```
MODULE greet()
OUTPUT "Welcome"
ENDMODULE
MODULE remind()
OUTPUT "Try an exercise"
ENDMODULE
BEGIN
CALL greet()
CALL remind()
END
```

Expected behaviour:
The main module calls greet and remind. Output: Welcome, then Try an exercise.

Review:
Identify two separate responsibilities.
Define the modules with clear names.
Draw the main module above the two called modules.

Exercise 120 Apply the concept

Main -> calculate -> validate. The links mean that one module calls another. They do not show Boolean branches, variable values or every execution step. validate need not be a direct child of Main.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 4 Parameters, arguments and scope

Exercise 121 Knowledge check 1

B. An argument
The actual supplied value is an argument; name is the parameter in the method definition.

Exercise 122 Knowledge check 2

C. A placeholder in a method definition
A placeholder in a method definition. Review the related theory and work through a small example to check the rule.

Exercise 123 Knowledge check 3

B. The region where a local identifier is accessible
The region where a local identifier is accessible. Review the related theory and work through a small example to check the rule.

Exercise 124 Knowledge check 4

A. The local parameter copy
The local parameter copy. Review the related theory and work through a small example to check the rule.

Exercise 125 Knowledge check 5

C. Argument count, order and compatible types
Argument count, order and compatible types. Review the related theory and work through a small example to check the rule.

Exercise 126 Find and fix the error

Line 5 - Syntax
CALL displayPair("Lebo", "Mpho")
The method expects two arguments. Supply both in the required order.

Exercise 127 Build or trace a solution

```
METHOD displayResult(name, mark)
OUTPUT name, mark
ENDMETHOD
BEGIN
CALL displayResult("Lebo", 68)
END
```

Expected behaviour:
Parameters: name and mark. Arguments: "Lebo" and 68. Output: Lebo, 68.

Review:
Declare placeholders in the method header.
Match the two actual arguments to their positions.
Do not confuse the local parameter names with caller variables.

Exercise 128 Apply the concept

The parameter receives a copy of the integer value. Reassigning that local parameter does not reassign the caller’s original variable. Returning a result or an explicitly different passing mode is needed to communicate the new value.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 4 Functions and return values

Exercise 129 Knowledge check 1

B. RETURN result
RETURN supplies the result to the caller; OUTPUT displays it.

Exercise 130 Knowledge check 2

B. No
No. Review the related theory and work through a small example to check the rule.

Exercise 131 Knowledge check 3

A. Supplies a result and exits that invocation
Supplies a result and exits that invocation. Review the related theory and work through a small example to check the rule.

Exercise 132 Knowledge check 4

C. Every normal finishing path
Every normal finishing path. Review the related theory and work through a small example to check the rule.

Exercise 133 Knowledge check 5

B. In an assignment or expression
In an assignment or expression. Review the related theory and work through a small example to check the rule.

Exercise 134 Find and fix the error

Line 3 - Logic
RETURN result
The function promises a returned value. Displaying the result alone does not fulfil that contract.

Exercise 135 Build or trace a solution

```
METHOD larger(a, b) RETURNS REAL
IF a &gt;= b THEN
RETURN a
ELSE
RETURN b
ENDIF
ENDMETHOD
BEGIN
SET result = larger(8, 5)
OUTPUT result
END

Expected behaviour:
larger(8,5) -&gt; 8; larger(3,9) -&gt; 9; larger(4,4) -&gt; 4.

Review:
State the return type and parameters.
Make every possible branch return a value.
The caller stores the returned value before displaying it.
```

Exercise 136 Apply the concept

```
METHOD isEven(number) RETURNS BOOLEAN
  RETURN number MOD 2 = 0
ENDMETHOD
Tests: 4 -&gt; TRUE; 5 -&gt; FALSE; 0 -&gt; TRUE. RETURN communicates the Boolean result; OUTPUT
alone would only display it.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.
```

Unit 4 Passing arrays to methods

Exercise 137 Knowledge check 1

A. Count may differ from capacity
The method may need to process only the meaningful portion of a partially filled array.

Exercise 138 Knowledge check 2

A. 0 <= count <= capacity
0 <= count <= capacity. Review the related theory and work through a small example to check the rule.

Exercise 139 Knowledge check 3

C. Yes
Yes. Review the related theory and work through a small example to check the rule.

Exercise 140 Knowledge check 4

B. Yes, the reference copy can refer to the same array
Yes, the reference copy can refer to the same array. Review the related theory and work through a small example to check the rule.

Exercise 141 Knowledge check 5

A. 0, 1, 2
0, 1, 2. Review the related theory and work through a small example to check the rule.

Exercise 142 Find and fix the error

Line 2 - Boundary
FOR i = 0 TO count - 1
A zero-based collection containing count values ends at count - 1.

Exercise 143 Build or trace a solution

```
METHOD totalValues(values, count) RETURNS REAL
SET total = 0
FOR i = 0 TO count - 1
SET total = total + values[i]
ENDFOR
RETURN total
ENDMETHOD
BEGIN
SET result = totalValues(marks, 3)
OUTPUT result
END
```

Expected behaviour:
For marks=[10,20,30,999], count=3, output 60. The spare capacity is ignored.

Review:
Use count rather than assuming capacity equals meaningful data.
Visit 0 through count - 1.
Return the accumulated total to the caller.

Exercise 144 Apply the concept

Document the element type, zero-based indexing convention, meaningful count, capacity, whether count may be zero and whether the method changes the caller’s elements. Require 0 <= count <= capacity and test those boundaries.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 4 Overloading and method contracts

Exercise 145 Knowledge check 1

B. Change the parameter types
A distinct parameter signature can distinguish an overload.

Exercise 146 Knowledge check 2

C. No
No. Review the related theory and work through a small example to check the rule.

Exercise 147 Knowledge check 3

B. No
No. Review the related theory and work through a small example to check the rule.

Exercise 148 Knowledge check 4

A. sum(a,b) and sum(a,b,c)
sum(a,b) and sum(a,b,c). Review the related theory and work through a small example to check the rule.

Exercise 149 Knowledge check 5

C. A closely related responsibility
A closely related responsibility. Review the related theory and work through a small example to check the rule.

Exercise 150 Find and fix the error

Line 4 - Syntax
Change the second header to calculate(REAL b) RETURNS REAL
The original parameter types and count are identical. Change the parameter type or count to create a distinct signature.

Exercise 151 Build or trace a solution

```
// METHOD display(INTEGER value)
// METHOD display(STRING value)
// CALL display(42) uses INTEGER.
// CALL display("42") uses STRING.

Expected behaviour:
42 is numeric; "42" is text. The parameter type distinguishes the signatures.

Review:
Compare parameter types, not parameter names.
Match each actual argument to the compatible signature.
Return type alone cannot distinguish an overload.
```

Exercise 152 Apply the concept

```
total(REAL first, REAL second) and total(REAL first, REAL second, REAL third) are
distinct signatures. total(REAL a, REAL b) and total(REAL x, REAL y) are not distinct
merely because parameter names differ.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.
```

Unit 5 Classes, objects and constructors

Exercise 153 Knowledge check 1

B. One particular Learner instance
An object is a specific instance with its own state.

Exercise 154 Knowledge check 2

B. An initial valid object state
An initial valid object state. Review the related theory and work through a small example to check the rule.

Exercise 155 Knowledge check 3

A. Yes
Yes. Review the related theory and work through a small example to check the rule.

Exercise 156 Knowledge check 4

C. Object state
Object state. Review the related theory and work through a small example to check the rule.

Exercise 157 Knowledge check 5

B. Behaviour associated with a class or module
Behaviour associated with a class or module. Review the related theory and work through a small example to check the rule.

Exercise 158 Find and fix the error

Line 4 - Logic
SET name = newName
The incoming value should initialise the attribute. Assigning name to itself ignores newName.

Exercise 159 Build or trace a solution

```
// CLASS Book
// - title : STRING
// - borrowed : BOOLEAN
// + borrow()
// + returnBook()
// bookA: title="Algorithms", borrowed=TRUE
// bookB: title="Databases", borrowed=FALSE

Expected behaviour:
Both objects share the Book definition but hold different attribute values.

Review:
Separate the type definition from its instances.
Give each instance its own state.
Use methods to express actions that belong to a book.
```

Exercise 160 Apply the concept

```
Book(newTitle) stores newTitle in title and initialises borrowed to FALSE. Validate that
the title is not empty. Two constructor calls create two distinct instances even if their
 titles are equal.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.
```

Unit 5 Encapsulation, inheritance and polymorphism

Exercise 161 Knowledge check 1

B. Course has learners
Containment suggests has-a, not an inheritance relationship.

Exercise 162 Knowledge check 2

A. Valid state through controlled access
Valid state through controlled access. Review the related theory and work through a small example to check the rule.

Exercise 163 Knowledge check 3

C. Is-a
Is-a. Review the related theory and work through a small example to check the rule.

Exercise 164 Knowledge check 4

B. Specialising an inherited operation with the same contract
Specialising an inherited operation with the same contract. Review the related theory and work through a small example to check the rule.

Exercise 165 Knowledge check 5

A. A common abstraction with specialised implementations
A common abstraction with specialised implementations. Review the related theory and work through a small example to check the rule.

Exercise 166 Find and fix the error

Line 2 - Design
PRIVATE balance AS REAL, with controlled access methods
Unrestricted public balance allows callers to bypass the deposit rules. Encapsulation controls updates through an appropriate interface.

Exercise 167 Build or trace a solution

```
// Person is a general type.
// Learner is a Person.
// Course has Learners.
// A Course is not a Learner.
```

Expected behaviour:
Use inheritance for the justified Learner-Person is-a relationship and association for Course-Learner.

Review:
Ask whether the specialised type fulfils the general contract.
Use has-a for a class containing or referring to other objects.
Keep private state protected by appropriate operations.

Exercise 168 Apply the concept

A Shape reference can refer to a Circle or Rectangle. Each overrides area() with its own implementation. The call uses the same operation signature; runtime object type determines the behaviour. Overloading instead distinguishes different parameter signatures.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Unit 5 Class diagrams and class organisation

Exercise 169 Knowledge check 1

B. Private visibility
A minus prefix denotes private visibility; a plus prefix denotes public visibility.

Exercise 170 Knowledge check 2

C. Public visibility
Public visibility. Review the related theory and work through a small example to check the rule.

Exercise 171 Knowledge check 3

B. Zero or more related objects
Zero or more related objects. Review the related theory and work through a small example to check the rule.

Exercise 172 Knowledge check 4

A. The general superclass
The general superclass. Review the related theory and work through a small example to check the rule.

Exercise 173 Knowledge check 5

C. Class diagram
Class diagram. Review the related theory and work through a small example to check the rule.

Exercise 174 Find and fix the error

Line 2 - Design
Point the hollow triangle to Account
The inheritance triangle points toward the more general superclass, Account.

Exercise 175 Build or trace a solution

```
// Library
// - name : STRING
// + borrow(book : Book) : BOOLEAN
// Book
// - title : STRING
// Association: Library 1 -- 0..* Book
```

Expected behaviour:
The library has a private name; borrow is public and returns a Boolean. Multiplicity 0..* permits no books or many.

Review:
Read the class name, attributes and operations separately.
Interpret visibility and return types.
Explain the association and its multiplicity without claiming an execution order.

Exercise 176 Apply the concept

Draw a solid line from SavingsAccount to Account with a hollow triangle pointing at Account, the general type. A procedural hierarchy chart is a different diagram and should not be substituted for a class diagram.

Review:
Check that your answer addresses every part of the question.
State assumptions and relevant boundary cases.
Equivalent correct solutions may use different names or structure.

Task library Methods

Exercise 177 Return the larger value

```
METHOD Maximum(first, second) RETURNS INTEGER
  IF first &gt;= second THEN
    RETURN first
  ELSE
    RETURN second
  ENDIF
ENDMETHOD
BEGIN
  OUTPUT Maximum(17, 9)
  OUTPUT Maximum(8, 8)
END
```

Check your result: 17 and 8.

Review:
Use METHOD or MODULE with a matching end.
Check the parameters, return value and call.
Trace normal, boundary and equal-value cases where relevant.

Exercise 178 A method that counts

```
METHOD DisplayNumbers()
  SET number = 1
  WHILE number &lt;= 5 DO
    OUTPUT number
    SET number = number + 1
  ENDWHILE
ENDMETHOD
BEGIN
  CALL DisplayNumbers()
END
```

Check your result: 1, 2, 3, 4, 5. The counter reaches 6 before the loop ends.

Review:
Use METHOD or MODULE with a matching end.
Check the parameters, return value and call.
Trace normal, boundary and equal-value cases where relevant.

Exercise 179 Rectangle area

```
METHOD RectangleArea(length, width) RETURNS REAL
  RETURN length * width
ENDMETHOD
BEGIN
  OUTPUT RectangleArea(6, 4)
  OUTPUT RectangleArea(0, 4)
END
```

Check your result: 24 and 0.

Review:
Use METHOD or MODULE with a matching end.
Check the parameters, return value and call.
Trace normal, boundary and equal-value cases where relevant.

Exercise 180 Input and average five marks

```
METHOD ReadAverage() RETURNS REAL
  SET total = 0
  FOR i = 1 TO 5
    INPUT mark
    SET total = total + mark
  ENDFOR
  RETURN total / 5.0
ENDMETHOD
BEGIN
  OUTPUT ReadAverage()
END
```

Check your result: For 50, 60, 70, 80, 90 the average is 70. Initialise total once before the loop.

Review:
Use METHOD or MODULE with a matching end.
Check the parameters, return value and call.
Trace normal, boundary and equal-value cases where relevant.

Exercise 181 RETURN or OUTPUT?

```
METHOD Square(number) RETURNS INTEGER
  RETURN number * number
ENDMETHOD
BEGIN
  SET result = Square(6)
  OUTPUT result
END
```

Check your result: 36. RETURN supplies a value to the caller; OUTPUT displays it.

Review:
Use METHOD or MODULE with a matching end.
Check the parameters, return value and call.
Trace normal, boundary and equal-value cases where relevant.

Exercise 182 Average through parameters

```
METHOD CalculateAverage(a, b, c) RETURNS REAL
  RETURN (a + b + c) / 3.0
ENDMETHOD
BEGIN
  OUTPUT CalculateAverage(60, 75, 90)
END
```

Check your result: 75. Use parentheses so addition happens before division.

Review:
Use METHOD or MODULE with a matching end.
Check the parameters, return value and call.
Trace normal, boundary and equal-value cases where relevant.

Exercise 183 A Boolean returning method

```
METHOD HasPassed(mark) RETURNS BOOLEAN
  RETURN mark &gt;= 50
ENDMETHOD
BEGIN
  IF HasPassed(48) THEN
    OUTPUT "Passed"
  ELSE
    OUTPUT "Failed"
  ENDIF
END
```

Check your result: Failed. Check marks 49, 50 and 51 to test the boundary.

Review:
Use METHOD or MODULE with a matching end.
Check the parameters, return value and call.
Trace normal, boundary and equal-value cases where relevant.

Exercise 184 Return a factorial

```
METHOD Factorial(number) RETURNS INTEGER
  SET product = 1
  FOR counter = 1 TO number
    SET product = product * counter
  ENDFOR
  RETURN product
ENDMETHOD
BEGIN
  OUTPUT Factorial(4)
  OUTPUT Factorial(0)
END
```

Check your result: 24 and 1. The FOR loop has no iterations for 0.

Review:
Use METHOD or MODULE with a matching end.
Check the parameters, return value and call.
Trace normal, boundary and equal-value cases where relevant.

Task library Overloading

Exercise 185 Overload by parameter count

```
METHOD Add(a AS INTEGER, b AS INTEGER) RETURNS INTEGER
  RETURN a + b
ENDMETHOD
METHOD Add(a AS INTEGER, b AS INTEGER, c AS INTEGER) RETURNS INTEGER
  RETURN a + b + c
ENDMETHOD

Add(4, 6) returns 10. Add(4, 6, 2) returns 12.

Review:
Differentiate the parameter count or types.
State the selected signature for each call.
Overload exercises are for written practice; duplicate method names are not supported by
the diagram lab.
```

Exercise 186 Overload by parameter type

```
METHOD Multiply(a AS INTEGER, b AS INTEGER) RETURNS INTEGER
  RETURN a * b
ENDMETHOD
METHOD Multiply(a AS REAL, b AS REAL) RETURNS REAL
  RETURN a * b
ENDMETHOD

Multiply(3, 4) returns 12. Multiply(2.5, 4.0) returns 10.0. Argument types select the
overload.

Review:
Differentiate the parameter count or types.
State the selected signature for each call.
Overload exercises are for written practice; duplicate method names are not supported by
the diagram lab.
```

Exercise 187 One name, two shapes

```
METHOD CalculateArea(side AS REAL) RETURNS REAL
  RETURN side * side
ENDMETHOD
METHOD CalculateArea(length AS REAL, width AS REAL) RETURNS REAL
  RETURN length * width
ENDMETHOD

CalculateArea(5.0) returns 25.0. CalculateArea(6.0, 4.0) returns 24.0.

Review:
Differentiate the parameter count or types.
State the selected signature for each call.
Overload exercises are for written practice; duplicate method names are not supported by
the diagram lab.
```

Exercise 188 Trace nested calls

Outputs: 5, 24, 8.
The first call has two arguments. The second has three. In the nested call the inner result is 3, so the outer two-argument call is Calculate(3, 5).

Review:
Differentiate the parameter count or types.
State the selected signature for each call.
Overload exercises are for written practice; duplicate method names are not supported by the diagram lab.

Exercise 189 Repair an invalid overload

Changing only the return type does not distinguish calls. Add a second parameter:

```
METHOD GetPrice(price AS REAL) RETURNS REAL
  RETURN price
ENDMETHOD
METHOD GetPrice(price AS REAL, discountPercent AS REAL) RETURNS REAL
  RETURN price - price * discountPercent / 100.0
ENDMETHOD
```

Assume a non-negative price and discount 0 to 100. GetPrice(200.0, 10.0) returns 180.0.

Review:
Differentiate the parameter count or types.
State the selected signature for each call.
Overload exercises are for written practice; duplicate method names are not supported by the diagram lab.

Task library Loops

Exercise 190 Count students who passed

```
BEGIN
  INPUT n
  SET count = 0
  FOR i = 1 TO n
    INPUT mark
    IF mark &gt;= 50 THEN
      SET count = count + 1
    ENDIF
  ENDFOR
  OUTPUT count
END

Equivalent WHILE solution:
BEGIN
  INPUT n
  SET count = 0
  SET i = 1
  WHILE i &lt;= n DO
    INPUT mark
    IF mark &gt;= 50 THEN
      SET count = count + 1
    ENDIF
    SET i = i + 1
  ENDWHILE
  OUTPUT count
END

For 45, 50, 82: 2 passed.

Review:
Initialise count before the loop.
Use identical bounds in both versions.
Increment the WHILE counter exactly once per iteration.
```

Exercise 191 Count even numbers

```
BEGIN
  INPUT n
  SET count = 0
  FOR i = 1 TO n
    INPUT number
    IF number MOD 2 = 0 THEN
      SET count = count + 1
    ENDIF
  ENDFOR
  OUTPUT count
END

Equivalent WHILE solution:
BEGIN
  INPUT n
  SET count = 0
  SET i = 1
  WHILE i &lt;= n DO
    INPUT number
    IF number MOD 2 = 0 THEN
      SET count = count + 1
    ENDIF
    SET i = i + 1
  ENDWHILE
  OUTPUT count
END

For -2, 0, 3, 8: 3 are even.

Review:
Initialise count before the loop.
Use identical bounds in both versions.
Increment the WHILE counter exactly once per iteration.
```

Exercise 192 Count salaries above a threshold

```
BEGIN
  INPUT n
  SET count = 0
  FOR i = 1 TO n
    INPUT salary
    IF salary &gt; 15000 THEN
      SET count = count + 1
    ENDIF
  ENDFOR
  OUTPUT count
END

Equivalent WHILE solution:
BEGIN
  INPUT n
  SET count = 0
  SET i = 1
  WHILE i &lt;= n DO
    INPUT salary
    IF salary &gt; 15000 THEN
      SET count = count + 1
    ENDIF
    SET i = i + 1
  ENDWHILE
  OUTPUT count
END

For 15000, 15001, 22000: 2 qualify.

Review:
Initialise count before the loop.
Use identical bounds in both versions.
Increment the WHILE counter exactly once per iteration.
```

Exercise 193 Calculate a class average

```
BEGIN
  INPUT n
  IF n &gt; 0 THEN
    SET total = 0
    FOR i = 1 TO n
      INPUT mark
      SET total = total + mark
    ENDFOR
    OUTPUT total / n
  ELSE
    OUTPUT "No students"
  ENDIF
END

WHILE conversion: set i = 1 before the loop, use WHILE i &lt;= n DO, keep the same body, add
 SET i = i + 1, then ENDWHILE. The average remains after the loop.

Review:
Avoid division by zero.
Accumulate every mark once.
For 50, 70, 90 the average is 70.
```

Exercise 194 Positive, negative or zero?

```
BEGIN
  INPUT n
  SET positive = 0
  SET negative = 0
  FOR i = 1 TO n
    INPUT value
    IF value &gt; 0 THEN
      SET positive = positive + 1
    ELSEIF value &lt; 0 THEN
      SET negative = negative + 1
    ENDIF
  ENDFOR
  OUTPUT positive, negative
END

WHILE conversion: initialise i to 1, test i &lt;= n, execute the same body and increment i
each time.
For -4, 0, 3, 7, -1: positive = 2, negative = 2.

Review:
Use two separate counters.
Do not classify zero as negative.
Keep input inside the loop.
```

Task library Parallel arrays

Exercise 195 Capture names and marks

```
BEGIN
  DECLARE names[5] AS STRING
  DECLARE marks[5] AS REAL
  FOR i = 0 TO 4
    INPUT names[i]
    INPUT marks[i]
  ENDFOR
  FOR i = 0 TO 4
    OUTPUT names[i], marks[i]
  ENDFOR
END

Review:
Both arrays have five elements.
The same index identifies one student in both arrays.
The valid indices are 0 through 4.
```

Exercise 196 Filter employee salaries

```
BEGIN
  DECLARE names[4] AS STRING
  DECLARE salaries[4] AS REAL
  FOR i = 0 TO 3
    INPUT names[i]
    INPUT salaries[i]
  ENDFOR
  FOR i = 0 TO 3
    IF salaries[i] &gt; 20000 THEN
      OUTPUT names[i], salaries[i]
    ENDIF
  ENDFOR
END

Review:
Use indices 0 through 3.
R20 000 does not qualify; R20 001 does.
Display the name and salary from the same index.
```

Exercise 197 Show all passing students

```
BEGIN
  SET names = ["Lebo", "Thabo", "Naledi", "Peter", "Maria"]
  SET marks = [65, 48, 72, 55, 81]
  FOR i = 0 TO 4
    IF marks[i] &gt;= 50 THEN
      OUTPUT names[i], marks[i]
    ENDIF
  ENDFOR
END

WHILE: initialise i = 0, loop while i &lt; 5, keep the IF body and increment i.
Output: Lebo 65; Naledi 72; Peter 55; Maria 81.

Review:
Visit all five indices.
Include exactly 50.
Preserve the name-mark pairing.
```

Exercise 198 Search by student name

```
BEGIN
  SET names = ["Lebo", "Thabo", "Naledi", "Peter", "Maria"]
  SET marks = [65, 48, 72, 55, 81]
  INPUT target
  SET foundAt = -1
  SET i = 0
  WHILE i &lt; 5 DO
    IF names[i] = target THEN
      SET foundAt = i
    ENDIF
    SET i = i + 1
  ENDWHILE
  IF foundAt &lt;&gt; -1 THEN
    OUTPUT names[foundAt], marks[foundAt]
  ELSE
    OUTPUT "Not found"
  ENDIF
END

Review:
Initialise foundAt to an invalid index.
Never read an array at index -1.
Test an existing name and a missing name.
```

Exercise 199 Find a product price

```
BEGIN
  SET names = ["Keyboard", "Mouse", "Monitor", "Printer", "Laptop"]
  SET prices = [250, 150, 2500, 1800, 9500]
  INPUT target
  SET foundAt = -1
  SET i = 0
  WHILE i &lt; 5 DO
    IF names[i] = target THEN
      SET foundAt = i
    ENDIF
    SET i = i + 1
  ENDWHILE
  IF foundAt &lt;&gt; -1 THEN
    OUTPUT names[foundAt], prices[foundAt]
  ELSE
    OUTPUT "Not found"
  ENDIF
END

Review:
Initialise foundAt to an invalid index.
Never read an array at index -1.
Test an existing name and a missing name.
```

Exercise 200 Find the top mark

```
BEGIN
  SET names = ["Lebo", "Thabo", "Naledi", "Peter", "Maria"]
  SET marks = [65, 48, 72, 55, 81]
  SET best = 0
  FOR i = 1 TO 4
    IF marks[i] &gt; marks[best] THEN
      SET best = i
    ENDIF
  ENDFOR
  OUTPUT names[best], marks[best]
END

Result: Maria, 81.

Review:
Initialise the best index to an existing element.
Use &gt; to preserve the first occurrence of a tie.
Use best to access both arrays.
```

Exercise 201 A complete results report

```
BEGIN
  SET names = ["John", "Mary", "David", "Sarah", "Peter"]
  SET marks = [45, 78, 66, 82, 39]
  SET passed = 0
  SET failed = 0
  SET best = 0
  FOR i = 0 TO 4
    OUTPUT names[i], marks[i]
    IF marks[i] &gt;= 50 THEN
      SET passed = passed + 1
    ELSE
      SET failed = failed + 1
    ENDIF
    IF marks[i] &gt; marks[best] THEN
      SET best = i
    ENDIF
  ENDFOR
  OUTPUT passed, failed
  OUTPUT names[best], marks[best]
END

Review:
Expected: 3 passed, 2 failed, Sarah 82.
Check that passed + failed equals 5.
Initialise counters outside the loop.
```

Task library Revision

Exercise 202 Build a salary flowchart

```
BEGIN
  INPUT basic
  INPUT sales
  INPUT rate
  SET salary = basic + sales * rate / 100.0
  OUTPUT salary
END

Review:
Expected salary: R6 000.
Use a parallelogram for input/output.
Use a process rectangle for the calculation.
```

Exercise 203 Stop on a sentinel

```
BEGIN
  INPUT number
  WHILE number &gt; 0 DO
    OUTPUT number * 3
    INPUT number
  ENDWHILE
END

Review:
Include the first input before the loop.
Read a new value each iteration.
For 2, 5, 0: display 6 and 15 only.
```

Exercise 204 Explain Boolean decisions

```
Eligible: isMember AND age &gt;= 65.
AND requires both conditions. OR would accept either condition, so it would not implement
 this rule. NOT reverses a Boolean result; NOT isMember is true for a non-member.
Both specified people are ineligible. A member aged 65 is eligible.

Review:
Check age 65 as the inclusive boundary.
Explain why OR changes the policy.
Use Boolean values rather than quoted strings for isMember.
```

Exercise 205 Return a practice contribution

```
METHOD Contribution(salary, rate) RETURNS REAL
  RETURN salary * rate / 100.0
ENDMETHOD
BEGIN
  OUTPUT Contribution(10000, 3)
END
```

```
Review:
Expected: 300.
The rate is passed as a parameter.
RETURN makes the result available to the caller.
```

Exercise 206 Plan before coding

```
Input: employee name and monthsRemaining.
Processing: leaveDays = monthsRemaining * 1.5.
Output: employee name and leaveDays.
Tests: 4 months -&gt; 6 days; 0 months -&gt; 0 days.
Validate monthsRemaining as an integer from 0 to 12 if validation is required.
```

```
Review:
Identify the formula separately from inputs.
Include units in the output.
Test zero months.
```

Task library OOP

Exercise 207 A class that prints numbers

```
CLASS NumberPrinter
  METHOD DisplayNumbers()
    FOR i = 1 TO 10
      OUTPUT i
    ENDFOR
  ENDMETHOD
ENDCLASS
BEGIN
  SET printer = NEW NumberPrinter()
  CALL printer.DisplayNumbers()
END

Review:
Separate the class definition from object use.
Check member visibility and loop bounds.
Class syntax is written practice; the flowchart converter does not parse classes.
```

Exercise 208 Even numbers through an object

```
CLASS EvenPrinter
  METHOD DisplayEven()
    SET number = 2
    WHILE number &amp;lt;= 20 DO
      OUTPUT number
      SET number = number + 2
    ENDWHILE
  ENDMETHOD
ENDCLASS
BEGIN
  SET printer = NEW EvenPrinter()
  CALL printer.DisplayEven()
END

Review:
Separate the class definition from object use.
Check member visibility and loop bounds.
Class syntax is written practice; the flowchart converter does not parse classes.
```

Exercise 209 A class with an array

```
CLASS MarkBook
  PRIVATE marks = [60, 75, 90]
  PUBLIC METHOD DisplayMarks()
    FOR i = 0 TO 2
      OUTPUT marks[i]
    ENDFOR
  ENDMETHOD
ENDCLASS
```

Encapsulation keeps marks private and exposes a public operation. A MarkBook object can call DisplayMarks().

Review:
Separate the class definition from object use.
Check member visibility and loop bounds.
Class syntax is written practice; the flowchart converter does not parse classes.

Exercise 210 Return a total from a class

```
CLASS NumberStore
  PRIVATE values = [4, 6, 10]
  PUBLIC METHOD Total() RETURNS INTEGER
    SET total = 0
    SET i = 0
    WHILE i &amp;lt; 3 DO
      SET total = total + values[i]
      SET i = i + 1
    ENDWHILE
    RETURN total
  ENDMETHOD
ENDCLASS
```

i is the index counter. total is the accumulator. The returned result is 20.

Review:
Separate the class definition from object use.
Check member visibility and loop bounds.
Class syntax is written practice; the flowchart converter does not parse classes.

Exercise 211 Model a person and student

```
CLASS Person
  PROTECTED address AS STRING
ENDCLASS
CLASS Student INHERITS Person
  PRIVATE number AS STRING
  PRIVATE firstName AS STRING
  PRIVATE surname AS STRING
ENDCLASS
```

Student is a Person. Draw a generalisation line from Student towards Person with a hollow triangle at Person. Protected allows direct subclass access; private restricts direct access to the declaring class. Prefer controlled methods when exposing data.

Review:
Separate the class definition from object use.
Check member visibility and loop bounds.
Class syntax is written practice; the flowchart converter does not parse classes.

Task library Arrays

Exercise 212 Visit every array element

```
BEGIN
  DECLARE values[5] AS INTEGER
  FOR i = 0 TO 4
    INPUT values[i]
  ENDFOR
  FOR i = 0 TO 4
    OUTPUT i, values[i]
  ENDFOR
END

Review:
The indices are 0, 1, 2, 3 and 4.
Input fills each element before it is displayed.
Use one array rather than separate variables.
```

Exercise 213 Array total and average

```
BEGIN
  SET values = [12, 8, 20, 10]
  SET total = 0
  FOR i = 0 TO 3
    SET total = total + values[i]
  ENDFOR
  OUTPUT total, total / 4.0
END

Review:
The total is 50 and the average is 12.5.
Initialise total once before the loop.
For an unknown length, check it is positive before division.
```

Exercise 214 Maximum in a single array

```
BEGIN
  SET values = [-8, -3, -12, -5]
  SET maximum = values[0]
  FOR i = 1 TO 3
    IF values[i] &gt; maximum THEN
      SET maximum = values[i]
    ENDIF
  ENDFOR
  OUTPUT maximum
END

Review:
Expected maximum: -3.
Starting at zero would give an incorrect result.
Assume a non-empty array.
```

Exercise 215 Find every matching index

```
BEGIN
  SET values = [4, 7, 4, 9, 4]
  INPUT target
  SET count = 0
  FOR i = 0 TO 4
    IF values[i] = target THEN
      OUTPUT i
      SET count = count + 1
    ENDIF
  ENDFOR
  OUTPUT count
END

Review:
For target 4: indices 0, 2, 4 and count 3.
A missing target gives count 0.
Continue searching after the first match.
```

Exercise 216 Reverse an array in place

```
BEGIN
  SET values = [2, 4, 6, 8, 10]
  SET left = 0
  SET right = 4
  WHILE left &amp;lt; right DO
    SET temp = values[left]
    SET values[left] = values[right]
    SET values[right] = temp
    SET left = left + 1
    SET right = right - 1
  ENDWHILE
  OUTPUT values
END

Review:
Expected: [10, 8, 6, 4, 2].
Stop when the indices meet or cross.
The middle element remains in place.
```

Exercise 217 Traverse a two-dimensional array

```
BEGIN
  DECLARE matrix[2][3] AS INTEGER
  FOR row = 0 TO 1
    SET rowTotal = 0
    FOR col = 0 TO 2
      INPUT matrix[row][col]
      SET rowTotal = rowTotal + matrix[row][col]
    ENDFOR
    OUTPUT row, rowTotal
  ENDFOR
END

Review:
Rows are 0 to 1 and columns 0 to 2.
Reset rowTotal at the start of each row.
Rows [1, 2, 3] and [4, 5, 6] give 6 and 15.
```

Task library IF / ELSE

Exercise 218 IF ELSE pass decision

```
BEGIN
  INPUT mark
  IF mark &gt;= 50 THEN
    OUTPUT "Pass"
  ELSE
    OUTPUT "Revise"
  ENDIF
END

Review:
49 gives Revise.
50 gives Pass.
Use ELSE for the alternative path.
```

Exercise 219 Odd or even

```
BEGIN
  INPUT number
  IF number MOD 2 = 0 THEN
    OUTPUT "Even"
  ELSE
    OUTPUT "Odd"
  ENDIF
END

Review:
0 is even.
8 is even and 7 is odd.
MOD returns the remainder.
```

Exercise 220 Validate a mark range

```
BEGIN
  INPUT mark
  IF mark &gt;= 0 AND mark &lt;= 100 THEN
    OUTPUT "Valid"
  ELSE
    OUTPUT "Invalid"
  ENDIF
END

Review:
0 and 100 are valid.
-1 and 101 are invalid.
Both bounds must hold at the same time.
```

Exercise 221 An ELSEIF grade ladder

```
BEGIN
  INPUT mark
  IF mark &gt;= 75 THEN
    OUTPUT "Distinction"
  ELSEIF mark &gt;= 50 THEN
    OUTPUT "Pass"
  ELSE
    OUTPUT "Revise"
  ENDIF
END

Review:
Test 49, 50, 74 and 75.
The first true branch runs.
Check the highest threshold first.
```

Exercise 222 Nested eligibility decisions

```
BEGIN
  INPUT age
  INPUT approved
  IF age &gt;= 16 THEN
    IF approved = TRUE THEN
      OUTPUT "Eligible"
    ELSE
      OUTPUT "Approval required"
    ENDIF
  ELSE
    OUTPUT "Too young"
  ENDIF
END

Review:
Aged 15: Too young.
Aged 16 and FALSE: Approval required.
Aged 16 and TRUE: Eligible.
```

Exercise 223 Calculate a conditional discount

```
BEGIN
  INPUT total
  INPUT isMember
  IF isMember AND total &gt;= 500 THEN
    SET due = total * 0.9
  ELSE
    SET due = total
  ENDIF
  OUTPUT due
END

Review:
Member spending 500 pays 450.
Non-member spending 500 pays 500.
Set due on both paths before OUTPUT.
```

Task library SWITCH / CASE

Exercise 224 SWITCH menu

```
BEGIN
  INPUT option
  SWITCH option
    CASE 1:
      OUTPUT "Add"
    CASE 2:
      OUTPUT "View"
    CASE 3:
      OUTPUT "Exit"
    DEFAULT:
      OUTPUT "Invalid option"
  ENDSWITCH
END

Review:
Match the selector against exact CASE values.
Use DEFAULT when no value matches.
Execute only the matching branch, then continue after ENDSWITCH.
```

Exercise 225 Day name lookup

```
BEGIN
  INPUT day
  SWITCH day
    CASE 1:
      OUTPUT "Monday"
    CASE 2:
      OUTPUT "Tuesday"
    CASE 3:
      OUTPUT "Wednesday"
    DEFAULT:
      OUTPUT "Unsupported day"
  ENDSWITCH
END

Review:
Match the selector against exact CASE values.
Use DEFAULT when no value matches.
Execute only the matching branch, then continue after ENDSWITCH.
```

Exercise 226 Text values in SWITCH

```
BEGIN
  INPUT size
  SWITCH size
    CASE "S":
      OUTPUT "Small"
    CASE "M":
      OUTPUT "Medium"
    CASE "L":
      OUTPUT "Large"
    DEFAULT:
      OUTPUT "Unknown size"
  ENDSWITCH
END

Review:
Match the selector against exact CASE values.
Use DEFAULT when no value matches.
Execute only the matching branch, then continue after ENDSWITCH.
```

Exercise 227 A small calculator

```
BEGIN
  INPUT a
  INPUT b
  INPUT operator
  SWITCH operator
    CASE "+":
      OUTPUT a + b
    CASE "*":
      OUTPUT a * b
    DEFAULT:
      OUTPUT "Unsupported operator"
  ENDSWITCH
END

Review:
Match the selector against exact CASE values.
Use DEFAULT when no value matches.
Execute only the matching branch, then continue after ENDSWITCH.
```

Exercise 228 Select a delivery charge

```
BEGIN
  INPUT zone
  SWITCH zone
    CASE 1:
      OUTPUT 30
    CASE 2:
      OUTPUT 50
    CASE 3:
      OUTPUT 80
    DEFAULT:
      OUTPUT "Invalid zone"
  ENDSWITCH
END

Review:
Match the selector against exact CASE values.
Use DEFAULT when no value matches.
Execute only the matching branch, then continue after ENDSWITCH.
```

Exercise 229 Trace a SWITCH decision

```
BEGIN
  INPUT option
  SWITCH option
    CASE 1:
      OUTPUT "Start"
    CASE 2:
      OUTPUT "Help"
    DEFAULT:
      OUTPUT "Try again"
  ENDSWITCH
END

Review:
Match the selector against exact CASE values.
Use DEFAULT when no value matches.
Execute only the matching branch, then continue after ENDSWITCH.
```